Reinforcement Learning-based Reliable Synthesis of Bioassays on MEDA Digital Microfluidic Biochips

DEBRAJ KUNDU, Electronic Design Automation, Technical University of Munich, Munich, Germany
GADIKOYILA SATYA VAMSI, KARNATI VIVEK VEMAN, GURRAM MAHIDHAR, and SUDIP ROY, Computer Science and Engineering, Indian Institute of Technology (IIT) Roorkee, Roorkee, India

Microfluidic biochips play an important role in point-of-care diagnosis. Advanced microfluidic biochips can efficiently perform various fluidic operations and robustly execute bioassays like protein synthesis, drug discovery, and many others. Micro-electrode-dot-array (MEDA) digital microfluidic biochips are one of the promising new generation microfluidic biochips consisting of an array of micro-electrodes with dedicated sensors on each electrode. Its ability to manipulate discrete droplets of different volumes and route them in any direction makes it an advanced microfluidic technology. However, the working principle of MEDA and conventional digital microfluidic biochips is based on the same phenomenon called electro-wetting-on-dielectric. In spite of all the advantages, MEDA still suffers from electrode malfunctioning due to dielectric breakdown. This kind of electrode malfunction happens primarily due to repeated actuation or prolonged actuation. Therefore, it is important to develop new bioassay synthesis methods that ensure reliability and guarantee error-free operations on MEDA. Since repeated and prolonged electrode actuations primarily occur within the modules, the reliability of MEDA biochips can be ensured with an appropriate placement strategy. As fluid routing is an inherent phase to complete the synthesis of any bioassay, we also propose a simple collision avoidance routing algorithm for MEDA. In this article, we propose a two-phase synthesis technique for MEDA biochips to improve the reliability of the biochips. First, a *reinforcement learning-based placement method* (*RLPM*) is designed for obtaining the reliability-aware placement of rectilinear-shaped microfluidic modules. Then, we propose a heuristic-based approach called *collision avoidance MEDA routing* (*CAMR*) that determines the collision-free routes for on-chip transportation of droplets of different volumes. *RLPM* utilizes the power of reinforcement learning and aims to minimize the utilization area of the chip while increasing its reliability. Whereas, *RLPM* together with *CAMR* minimizes the total completion time of the bioassay. Simulation results justify that the proposed synthesis technique can reduce the total chip area utilization by 28.6% (on average) without compromising the chip reliability compared to the state-of-the-art techniques.

CCS Concepts: • **Hardware → Electronic design automation**; **Emerging technologies**;

Additional Key Words and Phrases: biochips, digital microfluidics, MEDA biochips, mixing, sample preparation

The preliminary version submitted in ACM GLSVLSI, 2023.

Authors' Contact Information: Debraj Kundu (corresponding author), Electronic Design Automation, Technical University of Munich, Munich, Germany; e-mail: debraj.kundu@tum.de; Gadikoyila Satya Vamsi, Computer Science and Engineering, Indian Institute of Technology (IIT) Roorkee, Roorkee, India; e-mail: gvamsi@cs.iitr.ac.in; Karnati Vivek Veman, Computer Science and Engineering, Indian Institute of Technology (IIT) Roorkee, Roorkee, India; e-mail: kveman@cs.iitr.ac.in; Gurram Mahidhar, Computer Science and Engineering, Indian Institute of Technology (IIT) Roorkee, Roorkee, India; e-mail: gmahidhar@cs.iitr.ac.in; Sudip Roy, Computer Science and Engineering, Indian Institute of Technology (IIT) Roorkee, Roorkee, India; e-mail: sudip.roy@cs.iitr.ac.in.

This work is licensed under Creative Commons Attribution International 4.0.

© 2025 Copyright held by the owner/author(s).
ACM 2157-6912/2025/12-ART6
https://doi.org/10.1145/3773910

ACM Reference format:
Debraj Kundu, Gadikoyila Satya Vamsi, Karnati Vivek Veman, Gurram Mahidhar, and Sudip Roy. 2025. Reinforcement Learning-based Reliable Synthesis of Bioassays on MEDA Digital Microfluidic Biochips. *ACM Trans. Intell. Syst. Technol.* 17, 1, Article 6 (December 2025), 25 pages.
https://doi.org/10.1145/3773910

1 Introduction

Microfluidic biochips have been proved to be promising platforms for numerous biochemical applications (a.k.a. bioprotocols) such as prenatal and newborn screenings [11, 32], drug discovery [8], point-of-care medical diagnostics [4, 13, 31], and so on. Apart from the cost-effectiveness and portability of these microfluidic biochips, they also possess a bunch of advantages over lab-based macrosystems in terms of automation, accuracy, precision, and parallelization of different biochemical operations. In view of the global crisis due to the **Coronavirus disease (COVID-19)** pandemic, microfluidic biochips are also being used for the detection of Coronavirus [29].

In the last decade, advancement of the fabrication technologies for both **digital microfluidic biochips (DMFBs)** [1, 14, 33] and **continuous-flow microfluidic biochips (CFMBs)** [12] opens up a new horizon for research in this domain. A **micro-electrode-dot-array (MEDA)** biochip [14, 33] is an advanced DMFB chip, which consists of an array of micro-electrodes as shown in Figure 1(a), where each micro-electrode is integrated with a dedicated sensor and activation circuit. This makes MEDA biochips more flexible and error-resilient than conventional DMFBs. Moreover, a MEDA biochip can also realize rectilinear-shaped on-chip devices (referred to as modules) and any-angle routing of fluid droplets with different volumes. The working principle of both MEDA and conventional DMFBs for the manipulation of the droplets is based on the **electro-wetting-on-dielectric (EWOD)** mechanism [1]. Although a MEDA biochip has many advantages over conventional DMFBs, it still suffers from electrode degradation due to over actuations of the micro-electrodes. One on-chip module configured on a MEDA chip is shown in Figure 1(b), where some over-used micro-electrodes are shaded. It means that any further operation on these over-used micro-electrodes may jeopardize the reliability of the fluidic operation performed on the chip.

Due to the architectural differences of various microfluidic platforms (e.g., DMFBs, CFMBs), we need an architecture-specific synthesis methodology to efficiently realize any bioprotocol on the MEDA biochip. The synthesis of MEDA biochips primarily involves three phases: (i) scheduling of fluidic operations, (ii) fluidic module binding and placement, and (iii) fluid droplet routing. To date, several approaches have been proposed for one or more such phases [10, 16, 19, 23, 24, 26, 34]. However, the ***high-level synthesis for MEDA* (*HSM*)** [23] and the ***wear-leveling-based synthesis for MEDA* (*WLSM*)** [34] are the only two works to present the entire synthesis methodology for MEDA biochips. *HSM* [23] takes no measure to control the reliability of a MEDA biochip, while *WLSM* [34] considers reliability improvement as the primary objective. However, none of them aim to minimize the chip utilization area for obtaining the smallest size MEDA biochip, and neither of them considers the MEDA-specific rectilinear-shaped modules.

To bridge these research gaps, for a given bioprotocol, we propose a two-phase synthesis methodology considering the rectilinear-shaped modules that minimizes the chip utilization area of a MEDA biochip while maintaining the reliability of the chip. The key contributions of this work are as follows:

— to simultaneously improve the reliability and minimize the chip utilization area while determining the module placement and schedule on the MEDA biochip, we propose a

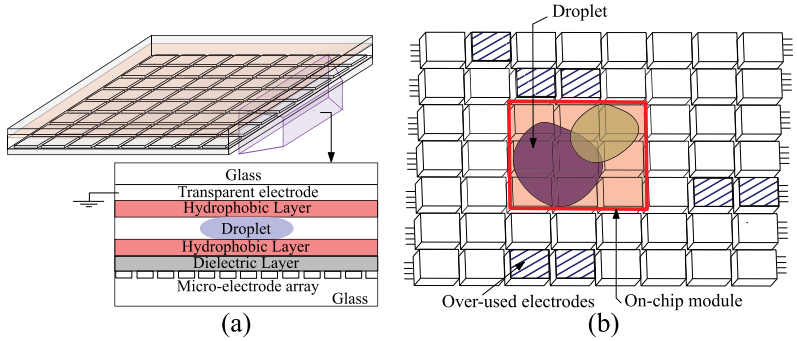

Fig. 1. A schematic diagram of MEDA architecture. (a) Cross-sectional view of different layers and (b) top view of a typical MEDA chip layout [14].

deep-Q-network (DQN) [28] based **reinforcement learning (RL)** approach called ***reinforcement learning-based module placement method* (*RLPM*)**;
- —the proposed *RLPM* considers rectilinear modules for placements on the MEDA biochip for the first time to determine the routing-aware placement; and
- —we also propose a priority-based routing method called ***collision avoidance MEDA routing* (*CAMR*)** to determine collision avoidance routing of droplets having different volumes while incorporating the droplet shape modulation and any-angle movement properties of MEDA biochips.

The remainder of the article is organized as follows. Section 2 describes the background, prior work, and motivation. In Section 3, the preliminaries and the overview of the work are illustrated. Section 4 and Section 5 describe the proposed approaches *RLPM* and *CAMR*, respectively. The complete synthesis methodology is explained in Section 6, and Section 7 presents the simulation results for performance evaluation compared to the state-of-the-art technique. Finally, Section 8 concludes the article.

2 Background, Prior Work, and Motivation

2.1 Background

The working principles of DMFBs and MEDA biochips are similar, as they follow the electromechanical process called *EWOD* [1, 30]. In EWOD, a droplet can be manipulated over a set of electrodes, and different types of on-chip fluidic operations, like dispensing, cutting, mixing, and so on, can be performed using on-chip modules defined by reconfiguring a set of electrodes. As shown in Figure 2(a), two groups of micro-electrodes are configured as two different types of mixers. A MEDA biochip specifically consists of a sea of identical micro-electrodes, each of which is around 10 times smaller than an electrode in conventional DMFBs [33]. This allows MEDA biochips to perform on-chip operations with droplets of different volumes, which is a bottleneck for conventional DMFBs. Due to such discrete manipulation of droplets and the advantage of diagonal movements, MEDA biochips can realize highly sophisticated mixers called **split-and-recombine (SAR)**-based laminar mixers [33], as shown in Figure 2(a). These mixers are capable of mixing droplets of larger volumes in relatively lesser operation time compared to that of any conventional circular mixer [2]. Additionally, instead of rectangular modules, we can also realize rectilinear modules in MEDA biochips [14]. Hence, we consider both rectangular and "L"-shaped modules for the synthesis of MEDA biochips. As an "L"-shaped mixer module is realized as a union of two

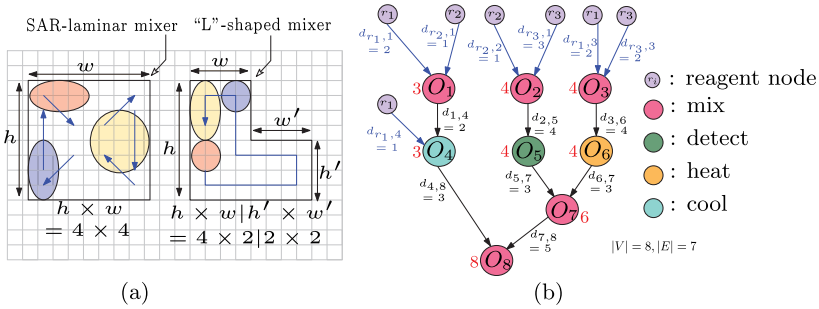

Fig. 2. (a) Modeling of on-chip mixers in MEDA biochip and (b) a sequencing graph representing a bioprotocol.

rectangular modules of sizes $h \times w$ and $h' \times w'$, respectively, we use the notation $h \times w|h' \times w'$ to denote an "L"-shaped module, as depicted in Figure 2(a).

A bioprotocol is generally represented as a sequencing graph with two types of nodes, namely input reagent nodes r_is and operation nodes O_is (Figure 2(b)). An incoming directed edge to an operation node O_p can come from another operation node O_j or from a reagent node r_i, indicating the dependency of that O_p, and the edge weight denotes the number of unit volumes of the fluid (reagent or intermediate) used in O_p. The edge weight from any node r_i to O_p is denoted as $d_{r_i,p}$ and the edge weight from any node O_i to O_p is denoted by $d_{i,p}$, where O_p is the child and O_i or r_i is the parent node. In Figure 2(b), $d_{r_i,p}$s are written in blue color (e.g., $d_{r_1,4} = 1$ and $d_{r_2,3} = 2$), $d_{i,p}$s are shown in black color (e.g., $d_{5,7} = 3$ and $d_{7,8} = 5$), and the total number of unit volumes of all the fluids used in any operation O_p is written beside the node in red color (e.g., O_5 and O_8 use a total of three and eight unit volumes of the fluids, respectively). Since the input reagents are stored in reservoirs, which are non-reconfigurable and static modules at the periphery of the chip, there is no need to schedule and to change the locations of the reagent nodes with time. So for the scheduling and placement phase, we consider a subgraph as the sequencing graph $G(V, E)$, which includes only the operation nodes and all the directed edges of black color as shown in Figure 2(b). Whereas, in the routing phase, all the information of the complete sequencing graph is used.

2.2 Prior Work

As MEDA biochip is the next generation of DMFBs with advanced functionalities, several research articles on its design automation were published during the last decade. Li et al. [23] proposed a method called *HSM* that targets the co-optimization of operation scheduling, module placement, and droplet routing. Since reliability is one of the primary concerns of any biochip, Zhong et al. [34] recently presented a reliability-aware synthesis methodology called *WLSM*. Further, Chung et al. [5] proposed a module placement algorithm for MEDA biochips that incorporates the completion-time uncertainties of on-chip operations. Several routing algorithms [19, 20, 22] for various biochips have been designed whose primary objective is to provide a collision-free routing solution. However, to optimize droplet routing on MEDA, in addition to the collision-free, we also must consider the droplet shape variations as one of the primary objectives. To optimize routing on MEDA biochips, various routing techniques have been designed [3, 10, 16, 17, 19, 24, 26]. Chen et al. [3] proposed a heuristic for collision avoidance routing of multiple droplets on MEDA chips. Keszocze et al. [19] presented a **satisfiability modulo theory (SMT)**-based exact routing algorithm for MEDA. Considering both splitting and reshaping of droplets on a MEDA biochip, a multi-level hierarchical approach-based routing method was described by Lu et al. [26]. Howladar et al. proposed two routing methods, out of which one [16] describes a priority-based homogeneous droplet routing

and another one [17] determines the contamination-aware routing paths by finding the best detours in the MEDA biochip. Furthermore, a multi-agent RL-based routing method was presented by Liang et al. [24] to increase the concurrent routing of the multiple droplets of the same volume. To overcome the erroneous operation of the digital biochip [18], a deep learning-based routing approach was presented. Again, to monitor the real-time health of the electrodes of a MEDA biochip, Liang et al. [25] and Elfar et al. [9] presented RL-based routing methods to increase the reliability of the chip.

In summary, researchers have been working towards improving the reliability of the MEDA biochips by proposing efficient techniques for different phases of MEDA biochip synthesis.

2.3 Motivation

An experimental study by Dong et al. [6] inferred that an electrode in a DMFB is completely degraded after it is actuated approximately over 200 times due to charge trapping. As a result, the force to move a droplet sitting on the degraded electrode decreases, and the droplet is likely to get stuck on that charge-trapped electrode. Moreover, excessive actuation of a set of electrodes can also lead to dielectric breakdown [7, 34], resulting in the malfunctioning of on-chip fluidic operations. The electrodes configured as a module are likely to be actuated more frequently than other electrodes, which are used only for droplet routing. So the reliability of a MEDA biochip is largely dependent on an effective placement of the modules on the chip. On the other hand, the chip utilization area is another important parameter, which needs to be minimized in order to design a small-sized portable microfluidic biochip. With a larger chip area, it is easier to place the modules without much overlapping among themselves, which in turn increases the reliability of the chip. Hence, the reliability increases if chip utilization area increases. Practically, it is desired to minimize the chip utilization area, whereas at the same time the reliability of the chip should be high. Note that none of the prior works has considered these two opposite objectives simultaneously. This motivates us to find an efficient placement method for MEDA biochips that can increase the reliability of the chip while minimizing the chip utilization area. However, in order to automate the loading of multiple reagents into the placed mixers, we need a routing algorithm. Since droplet shape morphing and splitting features can be utilized for routing of droplets on MEDA, this motivates us to develop a heuristic-based routing algorithm. The objective is to achieve a simpler and efficient synthesis of bioassays on MEDA through reliable mixer placement and shape-aware droplet routing.

3 Preliminaries and Overview

3.1 Preliminaries: Modeling of MEDA Biochip

Without loss of generality, we model a MEDA biochip as a 2-D grid $H \times W$ of height H and width W, where each grid-cell represents a micro-electrode. In order to avoid collisions with droplets and to maintain the fluidic constraints during the droplet routing phase, we consider two guard-rings around any on-chip module, namely the 1st guard-ring (one micro-electrode around the module) and the 2nd guard-ring (two micro-electrodes around the module) during the placement phase and the routing phase of the synthesis process, respectively (refer to Figure 3(a)).

In order to accurately model the synthesis problem for MEDA biochips, two unique properties of MEDA are considered: (i) manipulation of droplets of different volumes and (ii) droplet shape-changing ability, which includes both shape morphing and splitting. Due to the limitation of the EWOD force, we have to consider an upper bound of the droplet volume d. So in this work, the droplet volume d is chosen to be in the range of $[1, 8]$. When $d = 1$, it is referred to as a unit-droplet, which occupies the space of a unit-cell in such a way that no other droplet can reside in that unit-cell, as shown in Figure 3(b). In this work the size of a unit-cell is assumed to be 2×2, consisting of four micro-electrodes, and the volume of a unit-droplet ($d = 1$) is considered as the

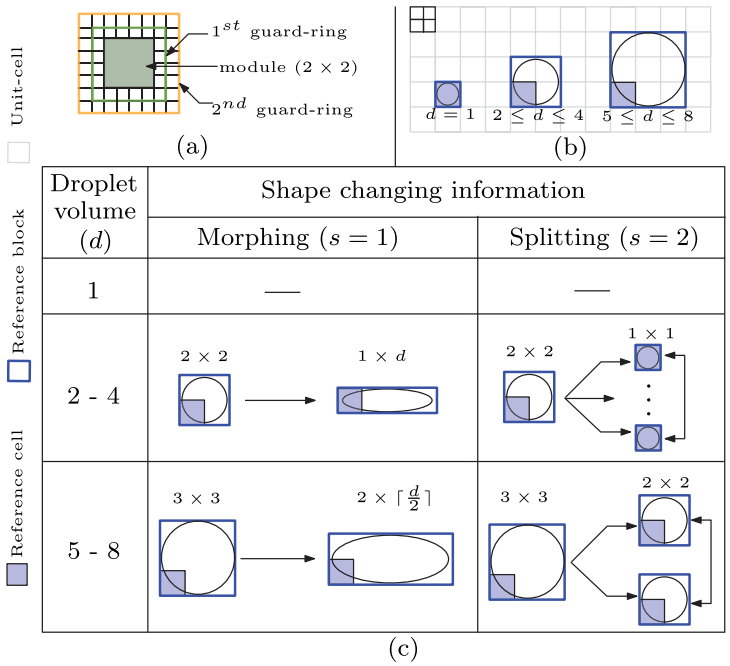

Fig. 3. (a) Two guard-rings around a module of size $h \times w$. (b) Reference blocks and reference cells for different volumes of droplet (d). (c) Droplet shape-changing parameter s.

capacity of a unit-cell. For any larger volume, i.e., $d > 1$, we generalize the shape of the droplet as a multiple of unit-cells referred to as reference block $h \times w$. For instance, if $2 \leq d \leq 4$, then the reference block is assumed to be of 2×2 unit-cells and if $5 \leq d \leq 8$, then it is assumed to be of 3×3 unit-cells, as shown in Figure 3(b). Moreover, in order to track a droplet during the routing phase, we consider a reference cell for each droplet, and these reference cells for different droplet volumes are shown in Figure 3(b). All the above assumptions are considered when the droplets are in normal shapes, i.e., without considering the changed shapes of the droplets. However, to include the droplet shape-changing properties, we consider some other assumptions, and all these assumptions are presented in Figure 3(c). Firstly, the shape of a unit-droplet can neither be changed nor can it be split into multiple smaller droplets, while any droplet of volume $d > 1$ can be morphed or split into multiple smaller droplets. If $2 \leq d \leq 4$, then it can either be morphed from 2×2 unit-cells to $1 \times d$ unit-cells, or it can be split into d unit-droplets. However, if $5 \leq d \leq 8$, then the droplet shape can either be morphed from 3×3 unit-cells to $2 \times \lceil \frac{d}{2} \rceil$ unit-cells or can be equally split into two droplets of 2×2 unit-cells. In Figure 3(c), the shape-changing information is referred as s, where $s = 1$ indicates droplet shape morphing and $s = 2$ implies droplet splitting.

Modules (devices) are configured using a set of electrodes on the biochip. As shown in Table 1, we consider a typical module library $\mathcal{M}$ having eight different types of modules, out of which the first five are mixer modules and the last three modules are for detection, heating, and cooling operations, respectively. In Table 1, the second column denotes the dimensions (sizes) of the modules in terms of $h \times w$ unit-cells, and the fourth column represents the total unit volume of the fluid (capacity) that a module can handle for its operation. The operation time (the last column in Table 1) for fluid mixing or detection depends on the module on which it is executed, whereas for heating and cooling the operation times are predefined in the sequencing graph of the bioprotocol.

Table 1. A Typical Module Library for MEDA Chip Synthesis

Module M_j	Size $h \times w$	Module Name	Capacity (Unit Vol.)	Operation Time
M_1	2×2	Mixer: circular	2	10
M_2	2×4	Mixer: circular	4	7
M_3	3×3	Mixer: SAR-laminar	4	3
M_4	4×4	Mixer: SAR-laminar	8	4
M_5	$4 \times 2 \vert 2 \times 2$	Mixer: L-circular	4	5
M_6	3×3	Detector	8	1
M_7	3×3	Heater	8	T_h [a]
M_8	3×3	Cooler	8	T_c [a]

[a] T_h and T_c are heating time and cooling time dependent on the bioprotocol.

3.2 Overview of the Synthesis Approach

The whole synthesis approach involves different phases like module binding, scheduling, placement, and routing.

3.2.1 Module Binding. For a given sequencing graph $G(V, E)$, we use a static module binding strategy to bind the modules with each of the operation nodes in G. In this work, we have chosen the sequencing graphs with four different types of operations: (i) mix, (ii) heat, (iii) cool, and (iv) detect, and a module library as shown in Table 1. We choose an iterative strategy for module binding, where at each iteration we select all the leaf operation nodes O_is, bind a suitable module M_j (from Table 1) with each of them, and then remove those O_is from G. In order to bind the modules from Table 1, we define the following selection rules. For all the mix operations with input size less than or equal to 4, we fix the selection priority of the modules as $< M_3, M_2, M_5, M_1 >$, and if the input size is greater than 4, then by default we have to select M_4. We have also assumed that except for M_1 and M_4, at most two consecutive instances of M_2, M_3, and M_5 can be used for binding. After the module binding, we label each O_i with the corresponding (O_i, M_j, T_i) tag, where M_j is a module of type j and T_i is the relative completion time for O_i. The module binding information $\widehat{B} = \bigcup_{\forall i \in V}(O_i, M_j, T_i)$ will be used in the successive schedule, placement, and routing phases.

3.2.2 Scheduling. Before module placement on the $H \times W$ grid, for scheduling the operation nodes we build two lists, namely the active list ($\mathcal{L}_1$) and the ready list ($\mathcal{L}_2$). Initially $\mathcal{L}_1$ is kept empty, but to fill $\mathcal{L}_2$ we select all the leaf nodes (O_is $\in V$) of G to calculate their priorities by the following equation:

$$S(O_i) = \alpha_1 \times \frac{1}{(\tilde{h} - \ell_i)} + \alpha_2 \times \frac{1}{|M_j|} + \alpha_3 \times \frac{1}{T_i},$$

where $\tilde{h}$ and ℓ_i ($0 \leq \ell_i \leq \tilde{h} - 1$) are the height of G and the level of O_i in G, respectively, and α_1, α_2 and α_3 are the parameters for the weights or importance to set the priorities of O_is, such that $0 \leq \alpha_1, \alpha_2, \alpha_3 \leq 1$ and $\alpha_1 + \alpha_2 + \alpha_3 = 1$. In order to place multiple modules simultaneously at a particular depth of G, we prioritize the O_is that can be bound with the modules of smaller size ($|M_j|$) and smaller operation time (T_j). Moreover, we also want to prioritize the O_is at lower depths of G. For instance, in this work, we consider $\alpha_1 = 0.4, \alpha_2 = 0.3$, and $\alpha_3 = 0.3$. Then, we insert the respective module binding information $(O_i, M_j, T_i) \in \widehat{B}$ into $\mathcal{L}_2$ in a descending order of the values of $S(O_i)$s. Next, after the module for an operation O_i is placed by the proposed method (*RLPM*), the corresponding $(O_i, M_j, T_i) \in \widehat{M}$ is removed from the list $\mathcal{L}_2$ and inserted in $\mathcal{L}_1$. Simultaneously $\mathcal{L}_2$ is also updated with all the module binding information of the child nodes of O_i.

3.2.3 Placement. In the placement phase, when a module is placed on a MEDA biochip, due to the reconfigurability nature of MEDA, some of the micro-electrodes may be over-utilized,

which degrades the reliability of the MEDA chip. So with the aim to generate a reliable synthesis methodology, we introduce a parameter called overlapping constraint $\alpha \in \mathbb{Z}^+$. For a given MEDA biochip of $H \times W$ size, the overlapping constraint α implies that a micro-electrode can be overlapped by at most α number of modules. Maintaining the given overlapping constraint of a MEDA biochip of $H \times W$ size, *RLPM* returns the placement and scheduling information $\widehat{P}$ for any O_i as $Place(O_i) = < |M_j|, (x_i, y_i), ts_i, te_i >$, where (x_i, y_i) is the top-left coordinate of the placed module M_j on the $H \times W$ grid, whereas ts_i, te_i are the starting and ending timestamps of O_i, respectively, such that $T_i = te_i - ts_i$.

3.2.4 Routing. After the completion of an operation O_i, i.e., after te_i timestamps, we need to route the droplet $d_{i,p}$ from (x_i, y_i) to (x_p, y_p) in the droplet routing phase. In some cases, if the child operation node O_p is not placed yet, then either we wait or store the droplet $d_{i,p}$ in a storage cell. In order to utilize all the capabilities of a MEDA biochip, we propose a priority-based routing method called *CAMR*. It determines the routing information denoted by $\widehat{R}$ that saves all the routing paths $Route(e_{i,p})$s for any subproblem generated by *RLPM* such that there are no unwanted collisions during the droplet routing phase.

4 RLPM

After obtaining the module binding information $\widehat{B}$ and initializing the active list $\mathcal{L}_1$ and ready list $\mathcal{L}_2$, we build a DQN-based method called *RLPM*, which efficiently places and schedules the modules specified in the ready list $\mathcal{L}_2$. The problem formulation for *RLPM* is as follows:

Inputs.

(1) The sequencing graph $G(V, E)$, the module library $\mathcal{M}$, and the corresponding module binding information $\widehat{B} = \bigcup_{\forall O_i \in V} (O_i, M_j, T_i)$.
(2) The active list $\mathcal{L}_1$ and the ready list $\mathcal{L}_2$.
(3) A MEDA biochip of $H \times W$ grid with an overlapping constraint α.

Outputs.

(1) The placement and scheduling information $\widehat{P} = \bigcup_{\forall i \in V} Place(O_i)$, where $Place(O_i) = < M_j, (x_i, y_i), ts_i, te_i >$.
(2) The total number of utilized cells U.
(3) The bounding-box area of the utilized cells $\mathcal{B}$.

Objectives. While maintaining the given overlapping constraint α, the primary objectives are as follows:

(1) Minimize the total place-and-schedule time $\mathcal{T}_p$ (defined later).
(2) Minimize the bounding-box area $\mathcal{B}$.
(3) Maximize the degree of reliability ρ (defined later).

4.1 Modeling for RLPM

The modeling of the proposed *RLPM* is explained through the building of the MEDA environment and the formulations of the state, action, and reward.

4.1.1 Building MEDA Environment. Considering the biochip layout as an array of $H \times W$ cells, the scheduling and placement information of module M_j with respect to operation O_i is stored as $Place(O_i) = < M_j, (x_i, y_i), (ts_i, te_i) >$. The set of utilized cells for M_j is denoted by $\mathcal{A}_i$ and referred to as the set of active-cells during the timestamp range (ts_i, te_i). For the reliability of the biochip,

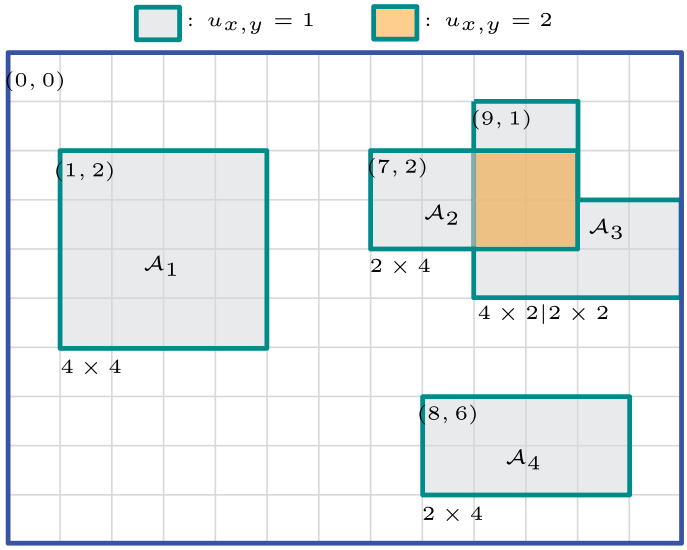

Fig. 4. A MEDA environment with four obstacles ($\mathcal{A}_1, \mathcal{A}_2, \mathcal{A}_3, \mathcal{A}_4$) having different overlapping values for the micro-electrodes.

we define the overlapping constraint (denoted as α), which implies the upper bound of the actuation count of a micro-electrode. We also define an overlapping value for each micro-electrode, denoted as $u_{x,y}$, which must be within the range $[0, \alpha]$. Here, $u_{x,y}$ corresponds to the electrode at position (x, y) on the $H \times W$ grid. So the placement constraints are: (i) modules should always be placed within the $H \times W$ grid, (ii) no active-cell should be utilized by more than one module, and (iii) $\forall(x, y), 0 \leq u_{x,y} \leq \alpha$. Based on these placement constraints at any timestamp t, the regions of the MEDA environment where any other module cannot be placed, i.e., $\cup_{\forall i}\mathcal{A}_i$, are called "obstacles," as shown in Figure 4. The cells with $u_{x,y} = \alpha$ are considered as "permanent obstacles," depicted as yellow regions in Figure 4.

4.1.2 State. A brute force way to define a *state* is to consider an image of the entire MEDA environment at a particular timestamp. This image should store the information of both free and obstacle regions within the current MEDA environment. A grid-based state representation will certainly be more informative, but the increased dimensionality of the state requires more training data and more computation, making the learning process inefficient or infeasible. Also, neural networks need to be larger and deeper; as a result, the training time and memory usage increase significantly. However, for a testcase with excessive sharing edges and a goal of routing-aware placement, we need to place the child and parent mixers as close as possible. In such complex cases, we need to consider grid-based state representation; otherwise, it will fail to achieve an optimal result. Since most of the real-life bioassays are not too complex, we choose a simple state representation. We have considered a *state* as s $= < \mathcal{B}, x, y >$, where $\mathcal{B}$ is the current bounding-box area and x, y is the position of the last module placed on MEDA. For instance, if M_i is placed at (x, y), we calculate the current bounding-box area $\mathcal{B}$, i.e., the area of the smallest rectangle, which includes all the placed modules. Let M_j be the next module to be placed; then the state corresponding to M_j will be $< \mathcal{B}, x, y >$.

In case we have some complex testcases, we may need to consider other possibilities of state representation, for example, (i) entire grid-based chip layout information, (ii) bounding-box area

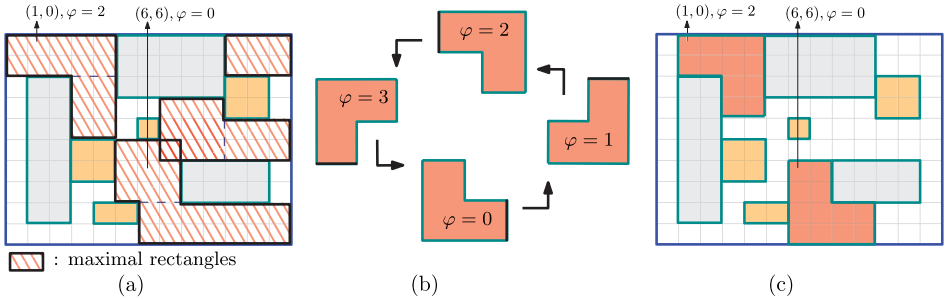

Fig. 5. (a) The "maximal rectilinear area" for a given number of obstacles. (b) All possible rectilinear orientations of an "L"-shaped module, and (c) possible positions for placement of this module in that "maximal rectilinear area."

$\mathcal{B}$ and collection of the (x, y) for all the previously placed mixers, and (iii) $\mathcal{B}$ and collection of cells reaching the overlapping threshold. For the first case, the problem size will increase, which increases the computation time. Compared with the proposed state $< \mathcal{B}, x, y >$, the last two state representations can capture more information about the layout that helps to avoid bad placements.

4.1.3 Action. In order to find a suitable place for M_j on $H \times W$ having obstacles, we find all the placeable maximal rectangles, and then we merge these rectangles having common boundaries to get the "maximal rectilinear polygons," as shown in Figure 5(a). Next, we check all possible placements for M_j within those polygons, which are considered possible actions for M_j.

While checking the placements for M_j, we rotate the module and consider all four rectilinear orientations of M_j. The number of rotations required to find any "candidate position" is symbolized as φ, where $\varphi \in \mathbb{Z}^+$ and $0 \leq \varphi \leq 3$ (e.g., Figure 5(b)). In the modeling for *RLPM*, *actions* are represented as $a_i =< x, y, \varphi >$, where (x, y) denotes the "candidate position" and φ implies the number of rotations required for the module to be placed at (x, y). For instance, if M_j of size $4 \times 2|2 \times 2$ needs to be placed on $H \times W$ at t, then all possible rotations of M_j are shown in Figure 5(b). As shown in Figure 5(c), there are two possible positions: (i) $(1, 0)$ with two rotations, i.e., $\varphi = 2$, and (ii) $(6, 6)$ with no rotation, i.e., $\varphi = 0$. So, the possible *actions* for the *agent* corresponding to the module $4 \times 2|2 \times 2$ are $< 1, 0, 2 >$ and $< 6, 6, 0 >$. In this problem, as the number of operations increases, the number of states and actions increases exponentially. Moreover, since the action space is discrete, we choose DQN to solve the problem. In general, the number of input and output neurons in a **deep neural network (DNN)** involved in DQN is determined according to the number of input features and the number of possible actions, respectively. However, for our placement problem, the number of "obstacles" varies with time. As a result, the possible set of *actions* dynamically changes for each module, and we cannot fix the number of output neurons. So, we consider a DNN as shown in Figure 6, where the input layer, i.e., the state-layer, has three neurons with respect to $< \mathcal{B}, x, y >$, the output layer, i.e., the action-layer, has one neuron, and in between it has two hidden layers with 256 and 128 neurons, respectively. For the *current-state* $< \mathcal{B}, x, y >$ and the module M_j, at first, we determine all possible *actions* and generate a DNN for each of those actions. Then for each *current-state* and *action* pair, the DNN predicts the corresponding Q-values. Next, we select the action corresponding to the maximum Q-value and update the *current-state* with the *next-state* corresponding to the selected *action* and repeat the same process for the new module to be placed.

4.1.4 Reward. After determining the *next-state* from the chosen *action*, we need to determine a reward to grade the *action*. The reward function is formulated in such a way that it encapsulates the parameters that ultimately help in achieving our objectives.

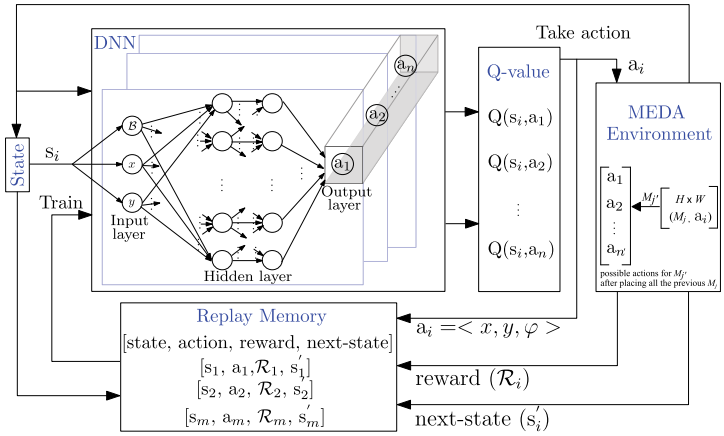

Fig. 6. DQN modeling for *RLPM*.

The 1st objective is to minimize the total place-and-schedule time $\mathcal{T}_p$, which is primarily dependent on the total number of scheduling steps. This time $\mathcal{T}_p$ can be reduced by maximizing the placement of modules in the ready list $\mathcal{L}_2$ (refer Section 3.2.2) at every timestamp. In general, if a module is placed towards the center of a MEDA biochip, it restricts the possibilities for placement of the next modules. As a result, multiple modules cannot be placed simultaneously at the same timestamp. Hence, to achieve the 1st objective, we defined the function $\mathcal{F}_1 = \frac{H}{(x^2+y^2+1)}$. This function increases when a module is placed towards the top-left corner of the chip, i.e., the cell $(0, 0)$. Next, to achieve the 2nd objective, i.e., to minimize the bounding-box area ($\mathcal{B}$) of the utilized cells, we consider two positive integers μ_1 and μ_2, such that $\mu_1 > \mu_2$ and $\mu_1\mu_2 = \sum_{\forall i \in \widehat{B}} |M_j|$, where $\sum_{\forall i \in \widehat{B}} |M_j|$ is an estimation of the total module area determined from the module binding information $\widehat{B}$. The difference between $\mu_1\mu_2$ and $\mathcal{B}$ is increased effectively with the help of (i) overlapping of the modules, maintaining the placement constraints, and (ii) finding the most suitable location for the modules in "maximal rectilinear polygons." In Figure 7, two different $\mathcal{B}$s are shown as the shaded regions for two separate actions. Clearly the action $< 1, 0, 2 >$ is better than $< 6, 6, 0 >$ as it minimizes the $\mathcal{B}$. To minimize $\mathcal{B}$, we define the function $\mathcal{F}_2 = \mu_1 - \frac{2\times\mathcal{B}}{\mu_2}$, which returns larger values when $\mathcal{B}$ is smaller than $\mu_1\mu_2$. Finally, for the 3rd objective to achieve a routing-aware placement, we define a function $\mathcal{F}_3 = \sum_{\forall e_{i,j} \in E} C_{i,j}$, where $e_{i,j}$ is the edge between O_i and O_j in the sequencing graph. $C_{i,j}$ is the number of overlapping electrodes between $\mathcal{A}_i$ and $\mathcal{A}_j$, i.e., $|\mathcal{A}_i \cap \mathcal{A}_j|$, where O_j is the parent node of O_i. This helps to minimize the distance between the placements of the parent and child operations and helps to simplify the routing complexities. Finally, we divide the sum of all the $\mathcal{F}_i$s with $\sum_{\forall(x,y)} 2^{u_{x,y}-\alpha}$ to ensure a reliable placement. For each $\mathcal{F}_i$ we assign a positive coefficient ω_i, which helps to set-up the relative importance among those three objective functions. Hence, the proposed reward function can be written as:

$$\mathcal{R} = \left[\omega_1 \times \frac{H}{(x^2+y^2+1)} + \omega_2 \times \left(\mu_1 - \frac{2\times\mathcal{B}}{\mu_2}\right) + \omega_3 \times \sum_{\forall e_{i,p} \in E} C_{i,p}\right] \times \frac{1}{\sum\limits_{\forall(x,y)} 2^{u_{x,y}-\alpha}}.$$

Based on the priorities of the three objectives, the values of ω_1, ω_2 and ω_3 need to be finalized. If scheduling time is the primary concern, we choose $(\omega_1 > \omega_3 > \omega_2)$. However, if we have a smaller chip area to implement a bioassay, we should choose $(\omega_2 > \omega_3 > \omega_1)$. Whereas, if we aim to achieve routing-aware placement, we should choose $(\omega_3 > \omega_1 > \omega_2)$. After some suitable iterations, when

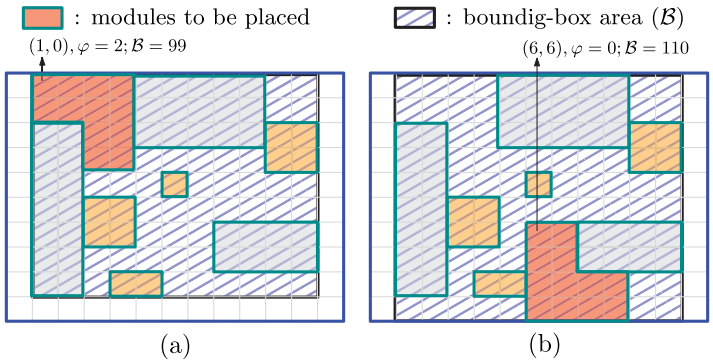

Fig. 7. $\mathcal{B}$ calculation for two different actions: (a) $< 1, 0, 2 >$ and (b) $< 6, 6, 0 >$.

the model learns and optimizes (maximizes) the value of the reward function, we get the placement and scheduling information for each O_i as $Place(O_i) =< M_j, (x_i, y_i), ts_i, te_i >$ recorded in $\widehat{P}$.

4.2 Evaluation of the Placement Parameters

To evaluate the performance of *RLPM*, we consider three parameters: (i) the total number of utilized cells (U), (ii) the bounding-box area ($\mathcal{B}$), and (iii) the degree of reliability (ρ). The total number of utilized cells U and the degree of reliability ρ are calculated as follows:

$$U = \Big| \bigcup_{\forall i \in V} \mathcal{A}_i \Big|, \tag{1}$$

$$\rho = \frac{U}{\sum\limits_{\forall (x,y) | u_{x,y} \leq \alpha} u_{x,y} + \sum\limits_{\forall (x,y) | u_{x,y} > \alpha} u_{x,y}^2}. \tag{2}$$

The value of ρ will be in the range of $(0, 1]$, where the higher value of ρ implies more reliable operations on the MEDA biochip. Once we get U, the bounding box $\mathcal{B}$ can be estimated as the minimum rectangular area that covers all the cells in U.

5 CAMR

After obtaining the placement and scheduling information $\widehat{P}$ by *RLPM*, we define the routing sub-problems $\mathbb{S}_\tau$ consisting of all the active operations till τ. For instance, Figure 8(a) shows M_js along with the start and end timestamps for the respective operations O_is. Here the end timestamps of O_1 and O_2 are 4, so there exists a routing sub-problem $\mathbb{S}_4$. As shown in Figure 8(b), the tasks suggested by $\mathbb{S}_4$ are to move the droplet $d_{1,4} = 3$ from the module of O_1 to that of O_4 and to move the droplet $d_{r,4} = 1$ from a reservoir to the module of O_4, and at the same time, to move the droplet $d_{2,5} = 4$ from the module of O_2 to that of O_5. In order to formalize this, one entity of the routing sub-problem $\mathbb{S}_\tau$ is denoted as $r_{i,p}$, which consists of $< (x_i, y_i), (x_p, y_p), d_{i,p}, \mathcal{A}_\tau >$, where (x_i, y_i) and (x_p, y_p) are the source-destination pair for the droplet of size $d_{i,p}$ and $\mathcal{A}_\tau$ is the set of active modules at timestamp τ. The primary challenge for routing of droplets is to maintain the fluidic constraints, i.e., no two droplets should touch each other during movement, and no route should pass through any active modules. Here, an active module $M_j \in \mathcal{A}_\tau$ is referred to as a module that is busy at timestamp τ. In case a droplet violates the fluidic constraints, then such an event is denoted as a *collision*. Based on the nature of the "obstacles," there are two possible types of collisions, namely **"route–module" collision (RMC)** and **"route–route" collision (RRC)**. Since we already

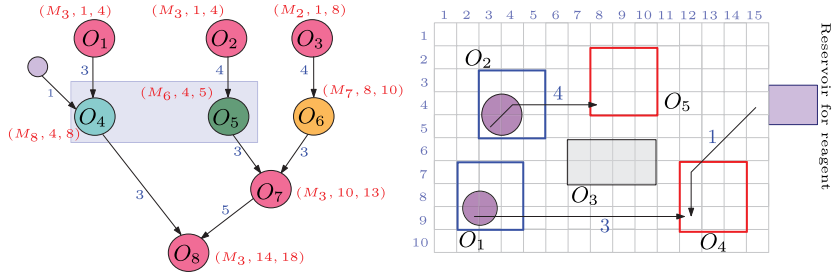

Fig. 8. (a) The placement and schedule information determined by *RLPM* for the sequencing graph G mentioned in Figure 2(b), and (b) the routing sub-problem at $\tau = 4$.

assumed a padding of unit-cells (guard-ring; Figure 3(a)) around each module, so a droplet of unit volume, i.e., $d_{i,p} = 1$, can always find an escape path, which ensures no route has to face a dead end.

5.1 Proposed Approach: CAMR

Initially, for each entity $r_{i,p}$ of a routing sub-problem $\mathbb{S}_\tau$ we determine the routing path $L_{i,p}$ using the A^* algorithm [15], which is a heuristic-based search technique. Due to its completeness and optimality, we consider A^* to find the initial routing paths, considering all the droplets to be of unit size and the active modules as obstacles. $L_{i,p}$ stores the sequence of cells returned by the A^* algorithm, which are considered as reference cells for the droplet of size $d_{i,p}$. Due to varying droplet size, A^* cannot avoid RMC or RRC trivially, so we modify the initial routing paths to determine the collision-free routing and store that modified route as $\widehat{L}_{i,p}$. An $\widehat{L}_{i,p}$ is initialized with $|L_{i,p}|$ number of elements, where its kth element stores the routing information at the kth cell. It is represented as $\widehat{L}^k_{i,p} = < L^k_{i,p}, d^k_{i,p}, s^k_{i,p}, t^k_{i,p} >$, where $L^k_{i,p}$ is the kth cell of $L_{i,p}$, $d^k_{i,p}$ denotes the droplet size, $s^k_{i,p}$ holds the shape-changing information, and $t^k_{i,p}$ indicates the stall period at kth cell. We initialize $d^k_{i,p}$, $s^k_{i,p}$, and $t^k_{i,p}$ as $d_{i,p}$, 0 and 1, respectively, and then modify them accordingly. Considering the above conditions, the droplet routing problem for the MEDA biochip is formulated as follows:

Input. Routing sub-problem $\mathbb{S}_\tau$, which stores the droplets to be routed at timestamp τ.

Output. Routing path $\widehat{L}_{i,p} = \{< L^0_{i,p}, d^0_{i,p}, s^0_{i,p}, t^0_{i,p} >, < L^1_{i,p}, d^1_{i,p}, s^1_{i,p}, t^1_{i,p} >, \cdots, < L^n_{i,p}, d^n_{i,p}, s^n_{i,p}, t^n_{i,p} >\}$.

Objective. Determine the *collision*-free routing paths.

Next, we discuss the proposed collision-avoidance techniques for RMC and RRC in detail.

5.1.1 RMC Avoidance. The initial route we get from the A* method avoids the obstacles, where the droplet size is assumed to be of unit size. However, due to varying droplet sizes, RMCs can be detected whenever a route passes closely from the side of a module. Considering the route position as the lower-left coordinate of the droplet, if there is any overlap between the droplet and a module, we identify it as RMC. If there is an RMC between a route $L_{i,p}$ and a module $M_j \in \mathcal{A}_\tau$, then to avoid this RMC we need to morph or split the droplet $d_{i,p}$ and update the $d^k_{i,p}$s and $s^k_{i,p}$s. The cell at which $L_{i,p}$ collides with M_j for the first time is called a "collision-state," and starting from the cell behind the collision-state, the sequence of cells through which this RMC persists is defined as the "collision-zone." This sequence of cells of $\widehat{L}_{i,p}$ in the collision-zone is stored in a list $\mathcal{Z}$. As shown in Figure 9(a), for the droplet $d_{i,p} = 2 \times 2$, $L_{i,p} = [(2, 9), (3, 8), (4, 7), \cdots, (8, 4), (8, 3), (8, 2)]$, where $(x_i, y_i) = (2, 9)$ and $(x_p, y_p) = (8, 2)$. The "collision-state" for $L_{i,p}$ is $(4, 7)$ as shown in Figure 9(b),

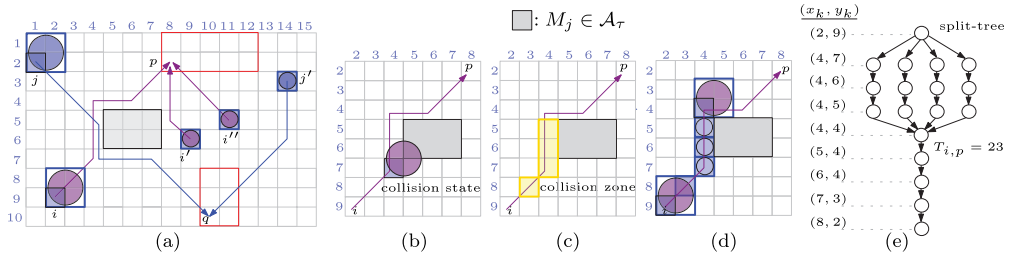

Fig. 9. (a) An instance of a routing sub-problem $\mathbb{S}_\tau$. (b) A "collision-state" and (c) the corresponding "collision-zone" for a droplet routing path $L_{i,p}$. (d) The modified droplet routing path after shape change and (e) the corresponding split-tree used for determining the modified route $\widehat{L}_{i,p}$ from $(2, 9)$ to $(8, 2)$.

and the corresponding "collision-zone" is shown in Figure 9(c). A part of the modified route $\widehat{L}_{i,p}$ is depicted in Figure 9(d), which shows the droplet shape-changing information in order to avoid this RMC. If the "collision-zone" is horizontal or vertical, then we morph; otherwise, we split the droplet. Before morphing or splitting of the droplet, we determine the maximum possible size of the droplet $\hat{d}_{i,p}$, which can pass through the "collision-zone," and decide the droplet shape-changing information using the table shown in Figure 3(c). In case of droplet splitting, each smaller droplet is routed sequentially, and to obtain the modified routes $\hat{L}_{i,p}$, $\hat{d}_{i,p}$ and $\mathcal{Z}$ are used to maintain a split-tree data structure as shown in Figure 9(e) (for the current example case shown in Figure 9(a)). This split-tree is denoted by $\mathcal{G}(\mathcal{V}, \mathcal{E})$, where $\mathcal{V}$ and $\mathcal{E}$ are the set of nodes and the set of directed edges, respectively. Each level of a split-tree represents the reference cell for the droplet, and the total number of nodes at each level denotes the number of droplets after the split. The total number of directed edges ($\mathcal{E}$) of all possible split-tree(s) of an $L_{i,p}$ is stored in $T_{i,p}$, which is an estimation of the routing timesteps required for the droplet $d_{i,p}$ to reach (x_p, y_p) while avoiding all the modules. The modified route $\widehat{L}_{i,p}$ can have multiple collisions with different modules, so even after avoiding the first RMC, we again need to check the remaining route until the droplet reaches (x_p, y_p). This approach for RMC avoidance is presented in the flowchart of *CAMR*, as shown in Figure 10.

5.1.2 RRC Avoidance. If there is an RRC between two or more routing paths, then we stall those routes based on the priorities of the respective paths to avoid that RRC. To do so, first, all the routing paths $L_{i,p}$s are divided into different groups g_ps based on their common destinations. All of the routes with destinations at (x_p, y_p) are contained in g_p, and we prioritize these groups according to their size, with the larger the group, the higher the priority. The priority list of these groups g_ps is denoted as P, which holds g_ps in a non-ascending. As the routes within a particular group g_p are moving towards the same destination, we assume that there will be no RRC between those routes. Hence, RRC can occur only between the routes from two different groups. Let $L_{i,p}$ and $L_{j,q}$ be two routes of g_p and g_q, respectively, such that g_p has higher priority than g_q. Then, in order to check whether RRC exists between $L_{i,p}$ and $L_{j,q}$, we (i) blow-up both the routes, considering the blow-up width as $d_{i,p}$ and $d_{j,q}$, respectively and (ii) label the sequence of cells of both the routes from the source to destination. In case the blown-up routes touch or overlap with each other, we define the collision-zone between those two routes as the union of (i) the set of cells in the overlapped region between the blown-up routes and (ii) the set of neighboring cells of the overlapped region, which strictly lies inside the blown-up regions. Let, $\mathcal{Z}_1$ and $\mathcal{Z}_2$ be the list of the sequence numbers of $L_{i,p}$ and $L_{j,q}$, respectively, inside the collision-zone. Now, if $\mathcal{Z}_1$ and $\mathcal{Z}_2$ have at least one common sequence number, then we claim that there is an RRC between $L_{i,p}$ and $L_{j,q}$. In order to avoid such RRC, we add a stall in the routing path from the group with

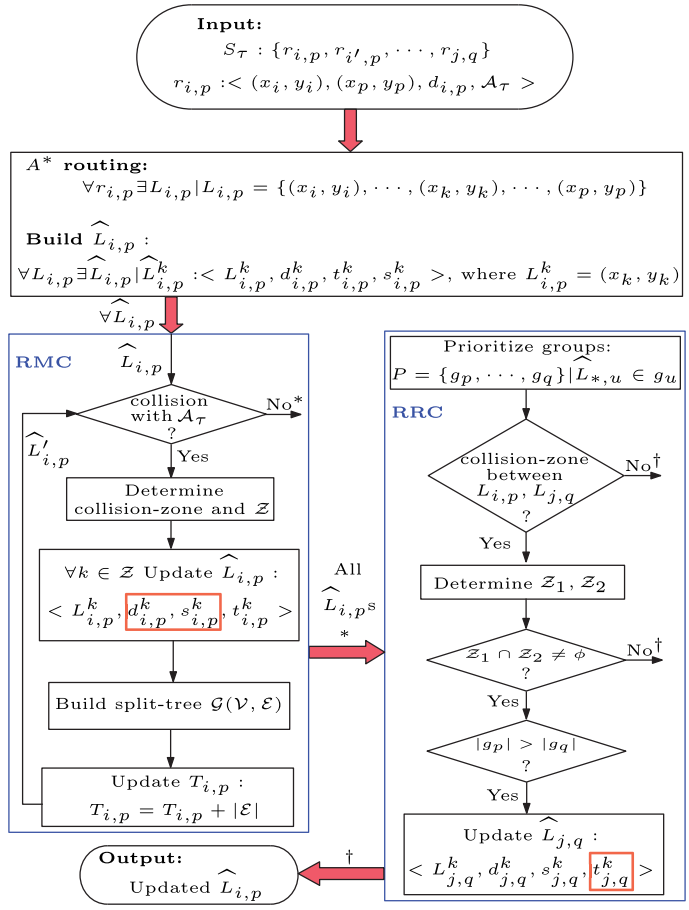

Fig. 10. Flowchart for the proposed approach *CAMR*.

lesser priority, i.e., $L_{j,q}$, and to impose a stall in $\widehat{L}_{j,q}$, we introduce the stall-state and the stall time δ for $L_{j,q}$ with respect to $L_{i,p}$. The cell (x_s, y_s) in $\widehat{L}_{j,q}$, where the droplet of size $d_{j,q}$ touches the collision-zone for the first time, is referred to as the stall-state. If the stall-state is the kth cell of $\widehat{L}_{j,q}$, then we update $t^k_{j,q}$ as $t^k_{j,q} + (max_{\forall \widehat{L}_{i,p} \in g_p}(T_{i,p}) - k)$. The blown-up routes of $L_{i,p}$ and $L_{j,q}$ in Figure 9(a) are shown in Figure 11(a), and the corresponding collision-zone is shown in Figure 11(b). Here, $\mathcal{Z}_1 = \{2, 3, 4, 5, 6, 7, 8\}$ (highlighted in magenta color) and $\mathcal{Z}_2 = \{3, 4, 5, 6, 7, 8\}$ (highlighted in black color) are the sub-sequences of $L_{i,p}$ and $L_{j,q}$, respectively. Since $\mathcal{Z}_1 \cap \mathcal{Z}_2 = \{3, 4, 5, 6, 7, 8\}$, which is not an empty set, two routes $L_{i,p}$ and $L_{j,q}$ collide with each other. As shown in Figure 9(a), there are three routes with destination (x_p, y_p) and two routes with destination (x_q, y_q), so $|g_p| = 3 > |g_q| = 2$. So, $L_{j,q}$ needs to be stalled to avoid this RRC. In Figure 11(b), the cell $(1, 2)$ highlighted in red color denotes the stall-state for $L_{j,q}$. A collision-zone can also exist between two non-intersecting , as shown in Figure 11(c), where the droplets are touching each other on their way of routing. Considering $|g_p| > |g_q|$, Figure 11(d) shows the corresponding collision-zone and the stall-state. The entire flow of *CAMR* is presented in Figure 10.

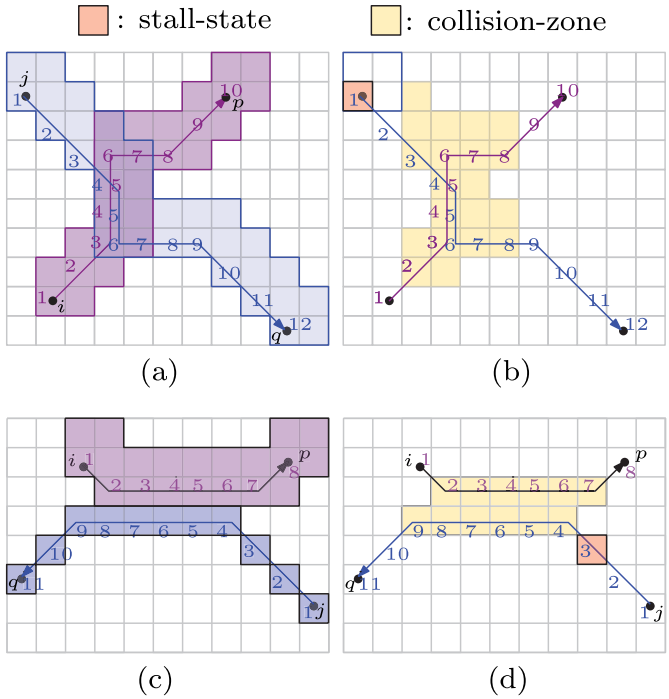

Fig. 11. Blown-up paths and the collision-zone for (a, b) intersecting routes and (c, d) non-intersecting routes.

5.2 Evaluation of the Routing Parameters

The *CAMR* incorporates all the unique properties of the MEDA biochip, i.e., shape morphing, splitting, and any-angle movement of the droplets, and generates collision avoiding routes. However, to evaluate the performance of *CAMR*, we consider three parameters: (i) the total route length ($\mathcal{L}_r$), (ii) the total route cost (C_r), and (iii) the total routing time ($\mathcal{T}_r$), which are calculated as follows:

$$\mathcal{L}_r = \sum_{\forall r_{i,p}} |\widehat{L}_{i,p}|, \tag{3}$$

$$C_r = \sum_{\forall r_{i,p}} \Big[\sum_{\forall d^k_{i,p} \in \widehat{L}_{i,p}} d^k_{i,p} \Big], \tag{4}$$

$$\mathcal{T}_r = max_{\forall r_{i,p}} \Big[T_{i,p} + \sum_{k=0}^{|\widehat{L}_{i,p}|} t^k_{i,p} \Big]. \tag{5}$$

6 RL-based Reliability-aware Synthesis: Entire Flow

Two methods *RLPM* and *CAMR*, discussed in the last two sections, are combined to obtain the complete synthesis flow. The entire flow of the proposed synthesis approach is shown in Figure 12. From the place-and-schedule phase, i.e., from *RLPM*, we get the set of routing sub-problems $\widehat{S} = \{\mathbb{S}_\tau, \cdots, \mathbb{S}_{\tau'}\}$s. Then in the routing phase, each routing sub-problem $\mathbb{S}_\tau$ is solved by the *CAMR* method as depicted by the flowchart of Figure 10. Generally, the mixers perform the most time-consuming operations (mixing), but an SAR-laminar mixer in a MEDA biochip can complete the operation much faster than any conventional circular mixer. Hence, in contrast to the assumptions

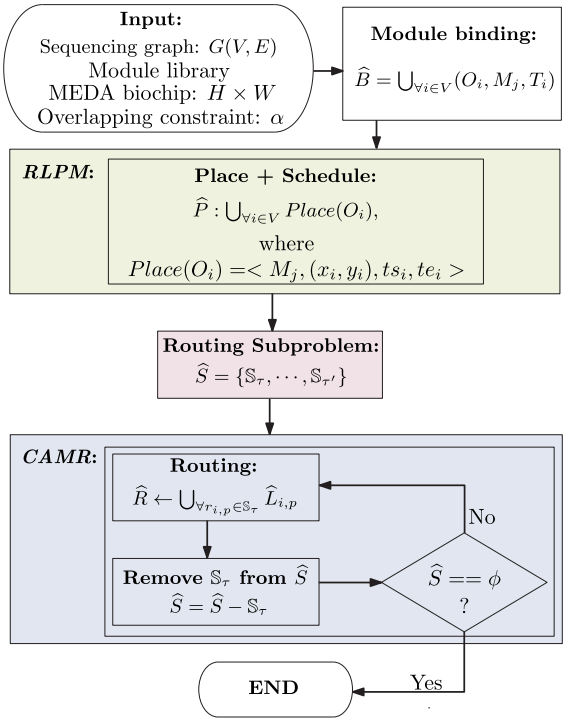

Fig. 12. Design flow for reliability-aware synthesis on MEDA.

made in prior work [2], it is true routing time is no longer negligible in the case of the MEDA biochip [23]. So, here we assume that within a MEDA chip of size $H \times W$, the time a droplet takes to travel $H + W - 1$ cells is the same as the operation time of a 3×3 SAR-laminar mixer (i.e., three unit time; refer to Table 1). Before computing the routing problem using *RLPM*, the total time for placement and scheduling of all the operation nodes in G is determined from $Place(O_i)$s as $\mathcal{T}_p = \forall_{Place(O_i) \in \widehat{P}}\, max(te_i)$. Therefore, the completion time of the entire synthesis flow $\widehat{T}$ can be estimated as $\widehat{T} = \lceil \mathcal{T}_p + \frac{3}{H+W-1} \times \mathcal{T}_r \rceil$.

7 Simulation Results

All simulations are performed on a computer with a 3.70 GHz Intel Xeon processor and 32 GB RAM running 64-bit Windows 10. We have considered seven bioprotocols as our testcases, which include both real-life and artificially designed bioprotocols. The details of these bioprotocols are shown in Table 2. For the simulation purpose, we consider the module library as mentioned in Table 1, and the size of the unit-cell is assumed as 2×2 micro-electrodes.

The hyperparameters considered during the training for *RLPM* are as follows: (i) learning rate is 0.9, (ii) batch size, i.e., the number of experiences sampled from the replay buffer for each training update of the neural network is 128, (iii) discount factor, which determines the importance of future rewards in the Q-value update is 0.9, (iv) initial value for ϵ is 1, in the ϵ-greedy exploration strategy, where the agent chooses a random action with probability ϵ and the final ϵ is $1e-3$, (v) decay rate for ϵ in ϵ-greedy exploration is 0.0003, (vi) number of epochs (episodes) over which ϵ-decays from initial epsilon to final epsilon is 1,500, (vii) total number of training episodes is 3,000, (viii) number of epochs between saving the model's weights is 1,000, (ix) size of the replay buffer, which stores

Table 2. Description of the Testcases

Testcase	Bioprotocol	#Operations	#Edges
T1	PCR [27]	7	6
T2	IVD [27]	12	6
T3	CPA [27]	55	54
T4	Synthetic Benchmark I	10	9
T5	Synthetic Benchmark II	20	19
T6	Synthetic Benchmark III	30	29
T7	Synthetic Benchmark IV (Figure 9(b) in [21])	9	10

CPA, Colorimetric Protein Assay; IVD, In-vitro Diagnostics; PCR, Polymerase Chain Reaction.

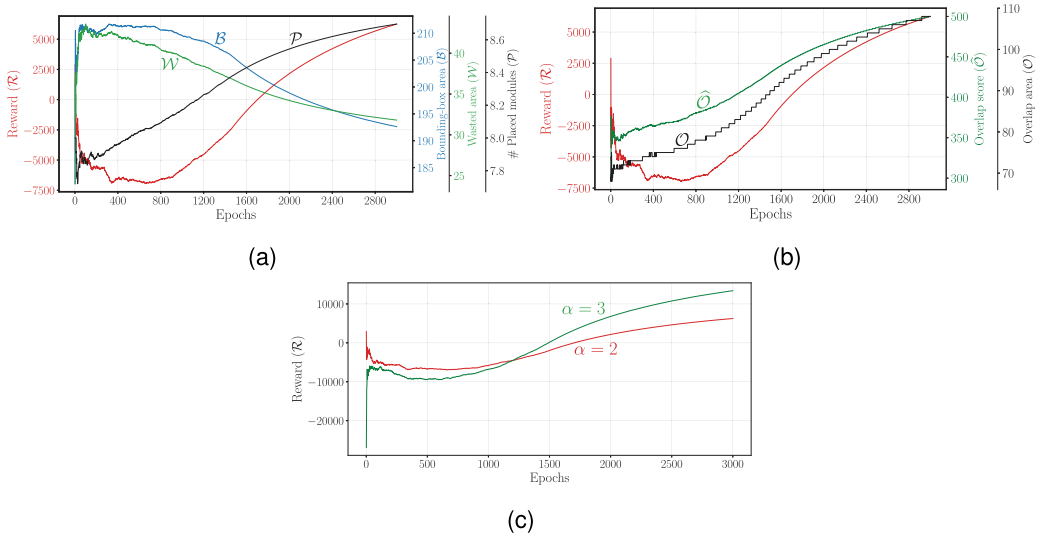

Fig. 13. For testcase T_6, the average distribution of the reward with respect to (a) $\mathcal{B}$, $\mathcal{W}$ and $\mathcal{P}$ and (b) the overlap score $\widehat{O}$ and overlap area O. (c) The average distributions of the rewards for $\alpha = 2, 3$.

experiences (state, action, reward, next state) for off-policy learning is 30,000, and (x) frequency (in epochs or steps) of updating the target network in DQN is 15. For all the testcases, we consider these parameters, although for smaller testcases we can reduce the number of training episodes.

7.1 Validation of RLPM

The validity and the performance of *RLPM* can be realized from the distribution of the variables mentioned in $\mathcal{F}_1$, $\mathcal{F}_2$ and $\mathcal{F}_3$ with respect to the distribution of the reward $\mathcal{R}$. In its initial epochs *RLPM* may determine some solution, which fails to place all the modules, maintaining the overlapping constraint of the MEDA biochip α. So in order to check whether *RLPM* learns from those mistakes and able to increase the number of placed modules with more number of epochs, we consider a variable $\mathcal{P}$ to represent the number of placed modules at each epoch. For testcase T6, Figure 13(a) depicts that the average values $\mathcal{B}$ and $\mathcal{W} = \mathcal{B} - U$ decrease and that of $\mathcal{P}$ and $\mathcal{R}$ increases with increasing number of epochs. As our primary objective is to minimize $\mathcal{B}$, the above result confirms that the reward function we propose performs quite well to achieve our objectives.

Moreover, with the aim to make *RLPM* a routing-aware placement method we allow the modules to be overlapped at different timestamps maintaining the overlapping constraint. We introduce two parameters, (i) overlapping score $\widehat{O} = |\bigcap_{\forall i \in \widehat{B}} \mathcal{A}_i|$ and (ii) overlapping area $O = |\bigcap_{\forall e_{i,j} \in E} \mathcal{A}_i|$, to verify their distributions with the rewards. For testcase T6, Figure 13(b) presents the distribution of the average values of $\widehat{O}$, O and $\mathcal{R}$ over 3,000 epochs. As $\widehat{O}$ and O increase with $\mathcal{R}$, we claim that *RLPM* is able to determine a routing-aware placement.

The overlapping constraint α is not included in the reward function $\mathcal{R}$, which makes sure that all the solutions of *RLPM* are subject to the constraint α. However, as α increases, the constraints become less strict and the problem becomes a bit easier for *RLPM* to solve. For testcase T6, Figure 13(c) depicts the distribution of $\mathcal{R}$ for two different scenarios, $\alpha = 2$ and $\alpha = 3$. Here, we can visualize that the learning rate for *RLPM* increases more rapidly after a certain number of epochs when α is changed from 2 to 3.

Again, to highlight the efficiency of *RLPM*, we present the variations of the parameters $\mathcal{R}$, $\mathcal{P}$, $\mathcal{B}$ and $\mathcal{W}$ for testcases T_7, T_2 and T_4 in Figure 16. For comparison, we fix both the chip size and α to 15×15 and 2 for all the testcases. For each case, we can observe that as the learning progresses, $\mathcal{R}$ and $\mathcal{P}$ increase, while $\mathcal{B}$ and $\mathcal{W}$ decrease based on the reward function. This proves that *RLPM* works efficiently for different testcases.

In order to validate the consistency of *RLPM*, in Figure 17(a) and (b) we present the deviation plots of the reward and the bounding-box area, respectively, considering the testcase T7 implemented on a MEDA biochip of size 15×15. For this problem instance, we run *RLPM* for 5 times each for 3,000 iterations. From the plots shown in Figure 17, we can verify that *RLPM* is consistent, as after certain iterations it always converges to optimize the values. It is to be noted that the computation time of *RLPM* depends on the size of the chip, the number of mixers to be placed, and the number of edges in the mixing graph. For these five simulations, the *RLPM* takes 0.8 hours, 1.52 hours, 0.57 hours, 0.86 hours, and 1.32 hours, respectively. Even though for most testcases we get the optimal results at around 1,400 iterations, for larger testcases we may need to run till 3,000 iterations to guarantee the best result. So, in the worst case, the *RLPM* takes around 1.5 hours to find the optimal results.

7.2 Performance Evaluation of RLPM + CAMR

However, we conclude that our proposed method is especially beneficial for determining a valid solution within a smaller chip size. Again considering a fixed chip size $H \times W = 30 \times 40$, we vary the overlap constraint α from 1 to 4 and present the distribution of U, $\mathcal{B}$, the routing cost $\mathcal{C}_r$, and the routing length $\mathcal{L}_r$ for *RLPM + CAMR* in Figure 18. Here, U and $\mathcal{B}$ reduce as α increases because *RLPM* has the opportunity to overlap more modules, which will saturate for some value of α. However, $\mathcal{C}_r$ and $\mathcal{L}_r$ tend to increase with α, because the routing length from the periphery to modules increases as bounding-box area decreases.

With the aim to compare the performance of our proposed approach, *RLPM + CAMR*, two of the most recent works: (i) *HSM* [23] and (ii) *WLSM* [34], are considered. Considering the testcase T4 mentioned in Table 2 and fixing $\alpha = 2$, the total cell utilization U, the bounding-box area $\mathcal{B}$ and the reliability metric ρ for *HSM*, *WLSM*, and *RLPM + CAMR* are compared in Figure 14(a)–(c). For chip size $H \times W = 26 \times 36$, *WSPM* could not find a valid solution; on the other hand, *HSM* and *RLPM* find a solution, but they are very unreliable as shown in Figure 14(c). As the chip size increases, *RLPM* is able to find valid solutions with lesser U (Figure 14(a)) and $\mathcal{B}$ (Figure 14(b)) compared to the state-of-the-art methods. In Figure 14(c), the reliability threshold is calculated as $\frac{H \times W}{\alpha \times \mathcal{B}}$, which is further utilized to depict the advantage of *RLPM* over *HSM* and *WLSM* by maintaining the reliability of the MEDA biochip for different chip sizes. For $H \times W = 38 \times 48$, all the methods are able to

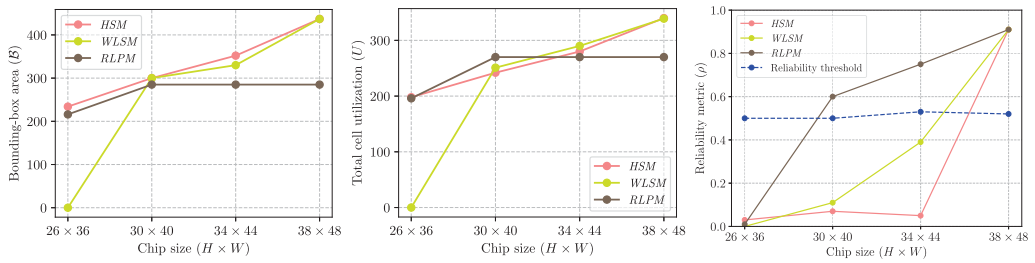

Fig. 14. Comparison of (a) bounding-box area $\mathcal{B}$, (b) total cell utilization U, and (c) reliability metric ρ for testcase T4 among *HSM*, *WLSM*, and *RLPM* over different chip sizes, with overlap constraint $\alpha = 2$.

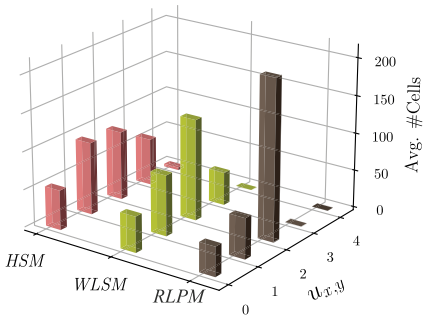

Fig. 15. Distribution of the average cell usage for *HSM*, *WLSM*, and *RLPM* + *CAMR* over all the testcases, where overlap constraint $\alpha = 2$.

determine valid solutions with a good reliability metric, because we have enough space to place the modules.

In another simulation, we evaluate the performances of *HSM*, *WLSM*, and *RLPM* + *CAMR* in maintaining the reliability of the chip, where we consider all the testcases with overlapping constraint $\alpha = 2$ and present the average number of cells having the same $u_{x,y}$s, as shown in Figure 15. This figure also suggests that for *RLPM* + *CAMR* none of the cells has $u_{x,y} \geq \alpha = 2$, so it proves that *RLPM* + *CAMR* is more reliable compared to *HSM* and *WLSM*.

Given a MEDA biochip, *WLSM* aims to place the modules in such a way that it expands the usage area to maintain the reliability of the biochip. However, *RLPM* simultaneously aims to minimize the usage area and maintain the reliability of the biochip. Due to these differences, we consider two separate cases to verify the effectiveness of our proposed approach compared to *HSM* and *WLSM*.

Case I. After determining the module binding information $\widehat{B}$ for each testcase, we calculate the total module area $\mathcal{A} = \sum_{\forall_i \in \widehat{B}} |\mathcal{A}_i|$. Then for each of those testcases, we choose the chip size as $H \times W$ for the smallest possible value of H, such that $H = W$ and $H^2 \geq \mathcal{A}$. Considering these setups, Table 3 shows the comparisons of U and $\mathcal{B}$ among the said methods. Among all the methods, only *RLPM* can handle rectilinear-shaped modules, so the total module area $\mathcal{A}$ for *RLPM* slightly varies from those for *HSM* and *WLSM* in some testcases. As shown in Table 3 for all the testcases, both U and $\mathcal{B}$ have been drastically reduced by the proposed method *RLPM*. On average *RLPM* manages to minimize both U and $\mathcal{B}$ by 23.45%, 27.63% and 28.60%, 32.49% compared to *HSM* and *WLSM*, respectively. These improvements prove that *RLPM* is highly effective for the synthesis of multiple

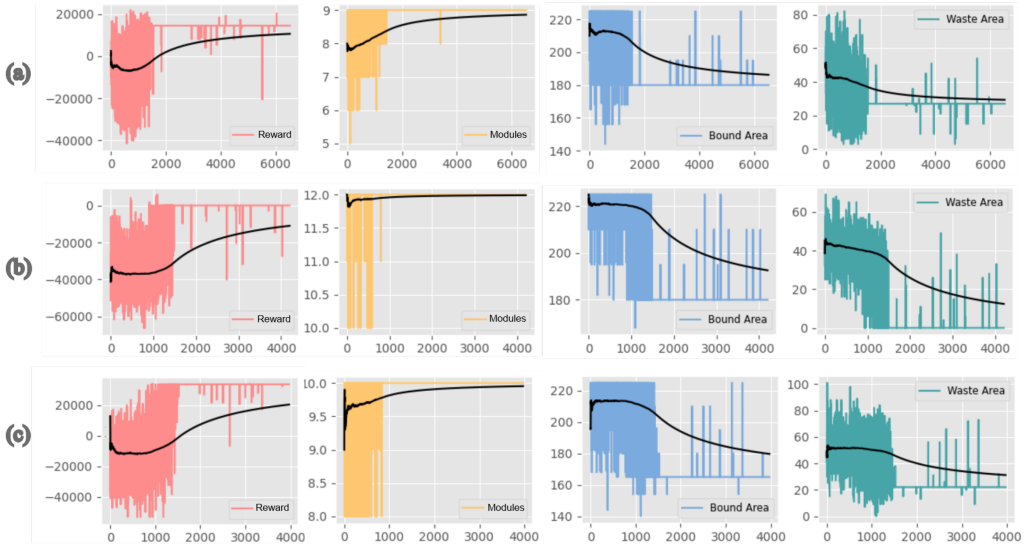

Fig. 16. Distribution of $\mathcal{R}$, $\mathcal{P}$, $\mathcal{B}$, and $\mathcal{W}$ after applying *RLPM* on (a) T_7, (b) T_2, and (c) T_4, where α and chip size are fixed as 2 and 15×15, respectively.

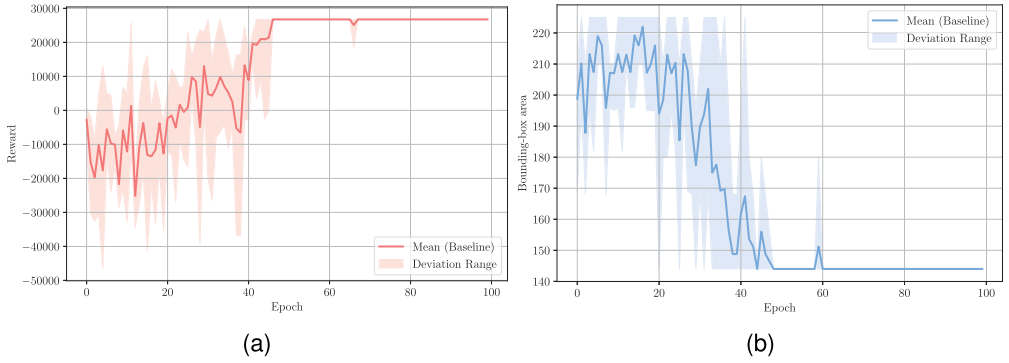

Fig. 17. Deviation plots of (a) reward $\mathcal{R}$ and (b) bounding-box area B after implementing T7 10 times with overlap constraint $\alpha = 2$ on a chip size of 10×10 and run for 3,000 epochs.

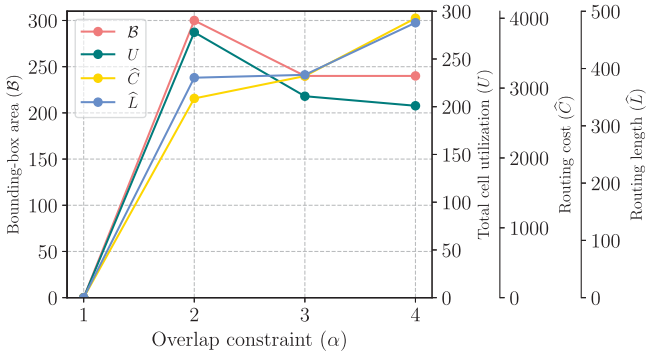

Fig. 18. Distribution of bounding-box area $\mathcal{B}$, total cell utilization U, routing cost C_r and routing length $\mathcal{L}_r$, for *RLPM* + *CAMR* with testcase T4, over different overlap constraint α when the chip size is fixed to 30×40.

Table 3. Comparison of Cell Utilization (U) and Bounding-box Area ($\mathcal{B}$) among *HSM* [23], *WLSM* [34], and *RLPM* for All the Testcases with Sufficiently Larger Chip Size

Testcase	Chip size	*HSM* [23]		*WLSM* [34]		*RLPM*			
	$H \times W$	U	$\mathcal{B}$	U	$\mathcal{B}$	U		$\mathcal{B}$	
T1	28 × 28	532	624	512	572	412		440 (20 × 22)	
						23.44%↓	**19.53%↓**	**29.49%↓**	**23.08%↓**
T2	34 × 34	816	960	888	1,024	688		768 (24 × 32)	
						15.69%↓	**22.52%↓**	**20.0%↓**	**25.0%↓**
T3	70 × 70	2,764	3,360	4,112	4,900	2,500		2,760 (46 × 60)	
						9.55%↓	**39.20%↓**	**17.86%↓**	**43.67%↓**
T4	32 × 32	748	832	748	832	560		560 (20 × 28)	
						25.13%↓	**25.13%↓**	**32.69%↓**	**32.69%↓**
T5	40 × 40	1,316	1,600	1,344	1,600	1,112		1,200 (30 × 40)	
						15.50%↓	**17.26%↓**	**25.0%↓**	**25.0%↓**
T6	54 × 54	2,092	2,484	2,400	2,916	1,564		1,620 (30 × 54)	
						25.24%↓	**34.83%↓**	**34.78%↓**	**44.44%↓**
T7	34 × 34	976	1,156	844	1,156	492		768 (24 × 32)	
						49.59%↓	**41.76%↓**	**33.56%↓**	**33.56%↓**
Average improvement percentage						**23.45%**	**28.60%**	**27.63%**	**32.49%**

The shaded regions highlight the improvement of RLPM w.r.t HSM and WLSM.

Table 4. Comparison of Cell Utilization, Route Cost, Route Length, Total Estimated Time and the Degree of Reliability among *HSM* [23], *WLSM* [34], and *RLPM + CAMR* for All the Testcases with Smaller Chip Size

Testcase	Chip size	*HSM* [23]					*WLSM* [34]					*RLPM + CAMR*				
	$H \times W$	U	C_r	$\mathcal{L}_r$	$\widehat{T}$	ρ	U	C_r	$\mathcal{L}_r$	$\widehat{T}$	ρ	U	C_r	$\mathcal{L}_r$	$\widehat{T}$	ρ
T1	20 × 22	432	488	61	17	0.63 (✓)	416	648	81	17	0.63 (✓)	412	392	49	14	0.64 (✓)
T2	24 × 32	672	896	112	18	0.72 (✓)	672	904	113	18	0.72 (✓)	688	560	75	15	0.72 (✓)
T3	46 × 60	2,488	11,500	1,524	57	0.02 (×)	2,564	10,716	1,351	60	0.05 (×)	2,500	10,240	1,408	51	0.57 (✓)
T4	20 × 28	536	812	127	25	0.13 (×)	536	660	107	32	0.17 (×)	560	976	142	24	0.59 (✓)
T5	30 × 40	968	2,920	454	29	0.07 (×)	1,004	2,732	399	26	0.11 (×)	1,112	2,444	370	24	0.60 (✓)
T6	30 × 54	1,392	6,200	844	40	0.04 (×)	1,416	5,708	815	36	0.04 (×)	1,564	5,460	742	35	0.57 (✓)
T7	24 × 32	612	1,108	202	16	0.13 (×)	636	756	151	16	0.31 (×)	492	640	102	14	0.69 (✓)
Average values			**3,417.71**	**474.86**	**28.86**	**0.25**		**3,160.57**	**431**	**29.29**	**0.29**		**2,958.86**	**412.57**	**25.29**	**0.63**
Improvement percentage													**13.43%**	**6.38%**	**13.12%**	**4.28%**	**12.37%**	**13.65%**	

The shaded regions highlight the improvement of RLPM+CAMR w.r.t HSM and WLSM.

bioprotocols simultaneously on a large MEDA biochip, as it can minimize the $\mathcal{B}$ for each of those bioprotocols.

Case II. For each testcase, considering the dimension of the bounding-box area $\mathcal{B}$ determined by *RLPM* as the chip size for the other two methods, we again compare all the three methods and present the corresponding placement and routing parameters in Table 4. From Table 4 we can see that both *HSM* and *WLSM* fail to maintain the reliability of the MEDA biochip as *RLPM + CAMR* does. Moreover, as shown in Table 4, the average decrement percentages of the total routing cost C_r by *RLPM + CAMR* are 13.43% and 6.38% compared to *HSM* and *WLSM*, respectively. Similarly, the total routing length $\mathcal{L}_r$ determined by *RLPM + CAMR* is also decreased by 13.12% and 4.28% compared to *HSM* and *WLSM*, respectively. Moreover, *RLPM + CAMR* manages to reduce the total completion time $\widehat{T}$ compared to the other methods, as shown in Table 4. Observing these improvements, we claim that *RLPM + CAMR* is highly effective as a reliable synthesis methodology while minimizing the area of utilization on a MEDA biochip.

Table 5. Comparison of Routing Lengths and Routing Time Required by the Baseline-route and *CAMR* for Routing Sub-problems

Subproblem Size	Droplet Size	Sources	Targets	Baseline-route		CAMR	
				$\mathcal{L}_r$	$\mathcal{T}_r$	$\mathcal{L}_r$	$\mathcal{T}_r$
3	1, 4, 9	[(0, 0), (3, 0), (7, 0)]	[(17, 7), (15, 17), (0, 16)]	236	100	61	30
	4, 4, 4	[(0, 0), (3, 0), (7, 0)]	[(16, 3), (16, 8), (16, 16)]	255	140	58	27
4	1, 4, 4, 6	[(0, 0), (3, 0), (5,0), (9, 0)]	[(17, 5), (17, 7), (15, 17), (0, 16)]	355	145	94	99
	1, 1, 4, 9	[(0, 0), (3, 0), (5, 0), (9, 0)]	[(17, 5), (17, 7), (15, 17), (0, 16)]	213	87	80	58
5	1, 1, 4, 4, 6	[(0, 0), (1, 0), (3, 0), (5, 0), (9, 0)]	[(17, 5), (17, 7), (15, 17), (16, 12) (0, 16)]	291	87	114	83
	1, 1, 4, 6, 9	[(0, 0), (1, 0), (3, 0), (5, 0), (9, 0)]	[(17, 5), (17, 7), (15, 17), (16, 12) (0, 16)]	388	116	118	88

To highlight the benefits of *CAMR*, we use a baseline routing method that cannot handle droplet shape morphing. This baseline approach, referred as Baseline-route, finds the route using A* algorithm and sequentially routes unit size droplets, avoiding any kind of collisions. Considering a chip of size 20×20 and obstacles at the following positions (1,10), (1,11), (1,12), (2,10), (2,11), (2,12), (3,10), (3,11), (3,12), (11,10), (11,11), (11,12), (12,10), (12,11), (12,12), (13,10), (13,11), (13,12), (11,3), (11,4), (11,5), (12,3), (12,4), (12,5), (13,3), (13,4), (13,5), (8,12), (8,13), (8,14), (9,12), (9,13), (9,14), (5,15), (11,8), we compare the routing length and the time required between the Baseline-route and CAMR, as shown in Table 5. It can be verified from this table that with the aid of the droplet shape morphing feature, the proposed *CAMR* outperforms the Baseline-route method.

8 Conclusions

This article discusses a complete synthesis flow for designing a MEDA-based **digital microfluidic (DMF)** biochip for the implementation of a bioprotocol. First we propose an *RLPM* utilizing a DQN-based modeling to achieve an efficient placement of rectilinear-shaped modules. Then, we introduce a priority-based routing method called *CAMR* on the MEDA biochip for determining the routes of droplets of different sizes, considering the droplet shape-changing capability of MEDA. The proposed approach determines the smallest possible chip utilization area of a MEDA biochip to implement a bioprotocol while maintaining the reliability of the chip. Hence, the proposed synthesis methodology can help to execute multiple bioprotocols in parallel on a given MEDA DMF biochip. In the future, we will consider more generalized shapes of the modules and improve the parallelism in the droplet routing to obtain an improved synthesis methodology for MEDA biochips.

References

[1] Jean Berthier. 2012. *Micro-Drops and Digital Microfluidics*. William Andrew Publishing.

[2] Krishnendu Chakrabarty and Fei Su. 2007. *Digital Microfluidic Biochips: Synthesis, Testing and Reconfiguration Techniques*. CRC Press.

[3] Zhongkai Chen, Daniel Hsiang-Yung Teng, Gary Chung-Jhih Wang, and Shih-Kang Fan. 2011. Droplet routing in high-level synthesis of configurable digital microfluidic biochips based on microelectrode dot array architecture. *BioChip Journal* 5, 4 (2011), 343–352.

[4] Curtis D. Chin, Tassaneewan Laksanasopin, Yuk Kee Cheung, David Steinmiller, Vincent Linder, Hesam Parsa, Jennifer Wang, Hannah Moore, Robert Rouse, Gisele Umviligihozo, et al. 2011. Microfluidics-based diagnostics of infectious diseases in the developing world. *Nature Medicine* 17, 8 (2011), 1015–1019.

[5] Wen-Chun Chung, Pei-Yi Cheng, Zipeng Li, and Tsung-Yi Ho. 2018. Module placement under completion-time uncertainty in micro-electrode-dot-array digital microfluidic biochips. *IEEE Transactions on Molecular, Biological and Multi-Scale Communications* 4, 4 (2018), 811–821.

[6] Cheng Dong, Tianlan Chen, Jie Gao, Yanwei Jia, Pui-In Mak, Mang-I Vai, and Rui P. Martins. 2015. On the droplet velocity and electrode lifetime of digital microfluidics: Voltage actuation techniques and comparison. *Microfluidics and Nanofluidics* 18, 4 (2015), 673–683.

[7] Antonis I. Drygiannakis, Athanasios G. Papathanasiou, and Andreas G. Boudouvis. 2009. On the connection between dielectric breakdown strength, trapping of charge, and contact angle saturation in electrowetting. *Langmuir: The ACS Journal of Surfaces and Colloids* 25, 1 (2009), 147–152.

[8] Shirit Einav, Doron Gerber, Paul D. Bryson, Ella H. Sklan, Menashe Elazar, Sebastian J. Maerkl, Jeffrey S. Glenn, and Stephen R. Quake. 2008. Discovery of a hepatitis C target and its pharmacological inhibitors by microfluidic affinity analysis. *Nature Biotechnology* 26, 9 (2008), 1019–1027.

[9] Mahmoud Elfar, Yi-Chen Chang, Harrison Hao-Yu Ku, Tung-Che Liang, Krishnendu Chakrabarty, and Miroslav Pajic. 2023. Deep reinforcement learning-based approach for efficient and reliable droplet routing on MEDA biochips. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 42, 4 (2023), 1212–1222.

[10] Mahmoud Elfar, Tung-Che Liang, Krishnendu Chakrabarty, and Miroslav Pajic. 2021. Formal synthesis of adaptive droplet routing for MEDA biochips. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 8 (2021), 2504–2517.

[11] H. C. Fan, Y. J. Blumenfeld, U. Chitkara, L. Hudgins, and S. R. Quake. 2008. Noninvasive diagnosis of fetal aneuploidy by shotgun sequencing DNA from maternal blood. *Proceedings of the National Academy of Sciences* 105, 42 (2008), 16266–16271.

[12] Luis M. Fidalgo and Sebastian J. Maerkl. 2012. A software-programmable microfluidic device for automated biology. *Lab on a Chip* 32, 6 (2012), 353–360.

[13] Vladimir Gubala, Leanne F. Harris, Antonio J. Ricco, Ming X. Tan, and David E. Williams. 2012. Point of care diagnostics: Status and future. *Analytical Chemistry* 84, 2 (2012), 487–515.

[14] B. Hadwen, G. R. Broder, D. Morganti, A. Jacobs, C. Brown, J. R. Hector, Y. Kubota, and H. Morgan. 2012. Programmable large area digital microfluidic array with integrated droplet sensing for bioassays. *Lab on a Chip* 12, 18 (2012), 3305–3313.

[15] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. 1968. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics* 4, 2 (1968), 100–107.

[16] Pampa Howladar, Pranab Roy, and Hafizur Rahaman. 2019. A high-performance homogeneous droplet routing technique for MEDA-based biochips. *ACM Journal on Emerging Technologies in Computing Systems* 15, 4 (2019), 1–37.

[17] Pampa Howladar, Pranab Roy, and Hafizur Rahaman. 2021. Droplet transportation in MEDA-based biochips: An enhanced technique for intelligent cross-contamination avoidance. *IEEE Transactions on Very Large Scale Integration Systems* 29, 7 (2021), 1451–1464.

[18] Tomohisa Kawakami, Chiharu Shiro, Hiroki Nishikawa, Xiangbo Kong, Hiroyuki Tomiyama, and Shigeru Yamashita. 2023. A deep reinforcement learning approach to droplet routing for erroneous digital microfluidic biochips. *Sensors* 23, 21 (2023), 8924.

[19] O. Keszöcze, Z. Li, A. Grimmer, R. Wille, K. Chakrabarty, and R. Drechsler. 2017. Exact routing for micro-electrode-dot-array digital microfluidic biochips. In *Proceedings of the 2017 22nd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 708–713.

[20] Debraj Kundu, Jitendra Giri, Sataru Maruyama, Sudip Roy, and Shigeru Yamashita. 2021. Fluid-to-cell assignment and fluid loading on programmable microfluidic devices for bioprotocol execution. *Integration* 78 (2021), 95–109.

[21] Debraj Kundu, Sudip Roy, Sukanta Bhattacharjee, Sohini Saha, Krishnendu Chakrabarty, Partha P. Chakrabarti, and Bhargab B. Bhattacharya. 2021. Mixing models as integer factorization: A key to sample preparation with microfluidic biochips. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 41, 3 (2021), 558–570.

[22] Guan-Ru Lai, Chun-Yu Lin, and Tsung-Yi Ho. 2018. Pump-aware flow routing algorithm for programmable microfluidic devices. In *Proceedings of the Design, Automation and Test in Europe Conference and Exhibition (DATE '18)*, 1405–1410.

[23] Z. Li, K. Y.-T. Lai, P. H. Yu, K. Chakrabarty, T. Y. Ho, and C. Y. Lee. 2017. Droplet size-aware high-level synthesis for micro-electrode-dot-array digital microfluidic biochips.*IEEE Transactions on Biomedical Circuits and Systems* 11, 3 (2017), 612–626.

[24] Tung-Che Liang, Jin Zhou, Yun-Sheng Chan, Tsung-Yi Ho, Krishnendu Chakrabarty, and Cy Lee. 2021. Parallel droplet control in MEDA biochips using multi-agent reinforcement learning. In *Proceedings of the 38th International Conference on Machine Learning (ICML)*, Vol. 139. PMLR, 6588–6599.

[25] Tung-Che Liang, Yi-Chen Chang, Zhanwei Zhong, Yaas Bigdeli, Tsung-Yi Ho, Krishnendu Chakrabarty, and Richard Fair. 2024. Dynamic adaptation using deep reinforcement learning for digital microfluidic biochips. *ACM Transactions on Design Automation of Electronic Systems* 29, 2 (2024), 1–24.

[26] G. R. Lu, B. B. Bhattacharya, T. Y. Ho, and H. M. Chen. 2018. Multi-level droplet routing in active-matrix based digital-microfluidic biochips. In *Proceedings of the 2018 23rd Asia and South Pacific Design Automation Conference (ASP-DAC)*. IEEE, 46–51.

[27] Wajid Hassan Minhass, P. Pop, and J. Madsen. 2012. *System-Level Modeling and Synthesis Techniques for Flow-Based Microfluidic Very Large Scale Integration Biochips*. Technical University of Denmark, Denmark.

[28] Volodymyr Mnih, Koray Kavukcuoglu, David Silver, Alex Graves, Ioannis Antonoglou, Daan Wierstra, and Martin Riedmiller. 2013. Playing Atari with deep reinforcement learning. arXiv:1312.5602. Retrieved from https://arxiv.org/abs/1312.5602

[29] Hsuan-Yu Mu, Yu-Lun Lu, Tzu-Hung Hsiao, and Jen-Huang Huang. 2020. Microfluidic-based approaches for COVID-19 diagnosis. *Biomicrofluidics* 14, 6 (2020), 061504.

[30] Joo Hyon Noh, Jiyong Noh, Eric Kreit, Jason Heikenfeld, and Philip D. Rack. 2011. Toward active-matrix lab-on-a-chip: Programmable electrofluidic control enabled by arrayed oxide thin film transistors. *Lab on a Chip* 12, 2 (2011), 353–360.

[31] Shivangi Sachdeva, Ronald W. Davis, and Amit K. Saha. 2021. Microfluidic point-of-care testing: Commercial landscape and future directions. *Frontiers in Bioengineering and Biotechnology* 8 (2021), 1537.

[32] Rama S. Sista, Rainer Ng, Miriam Nuffer, Michael Basmajian, Jacob Coyne, Jennifer Elderbroom, Daniel Hull, Kathryn Kay, Maithri Krishnamurthy, Christopher Roberts, et al. 2020. Digital microfluidic platform to maximize diagnostic tests with low sample volumes from newborns and pediatric patients. *Diagnostics* 10, 1 (2020), 21.

[33] G. Wang, D. Teng, and Shih-Kang Fan. 2011. Digital microfluidic operations on micro-electrode dot array architecture. *IET Nanobiotechnology* 5, 4 (2011), 152–160.

[34] Zhanwei Zhong, Tung-Che Liang, and Krishnendu Chakrabarty. 2020. Enhancing the reliability of MEDA biochips using IJTAG and wear leveling. *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems* 39, 12 (2020), 2063–2076.

Received 25 April 2025; revised 6 September 2025; accepted 30 September 2025